Weekly Work Report – DevOps Intern

Week 3

Name: Kailas R Pillai

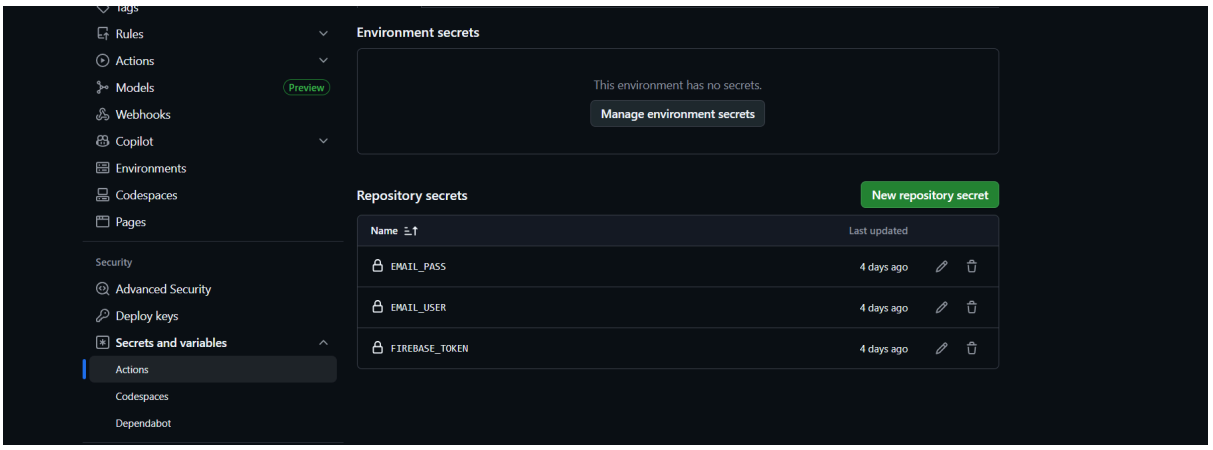
Day 1 - 29/12/2025

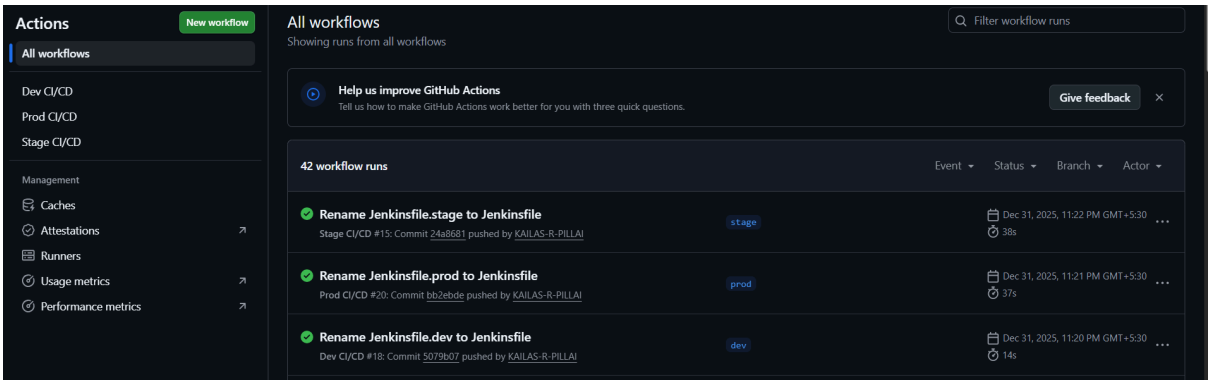
Tasks Completed Today:

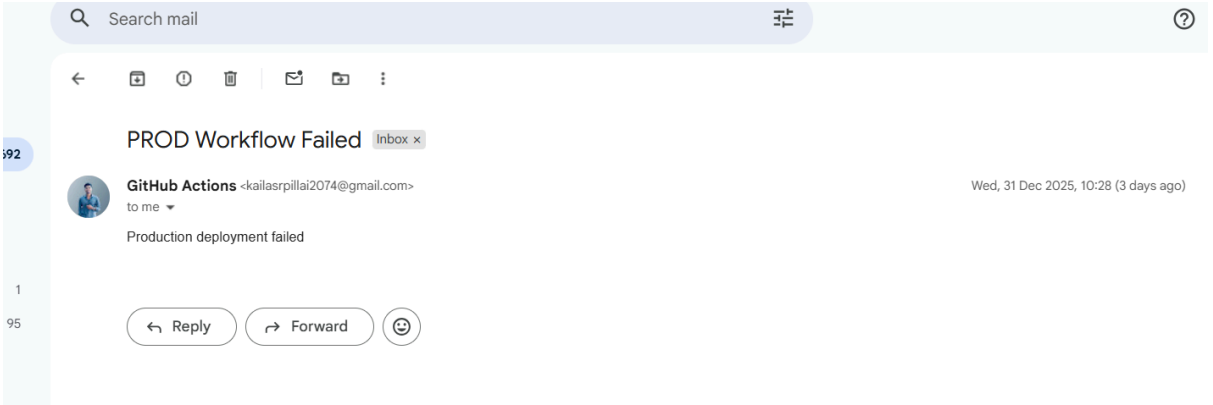
- **Task1:** Learned Github Actions, created CI/CD, used caching, created secrets, sent mail when build fails, created a repo with dev, stage, prod branches, created CI/CD for each branch.
- **Task2:** Learned git commands: branch, pull, add, commit, fetch.Learned the difference between merge and stash.
- **Task3:** Learned git branching strategies.

Learnings:

- **Github Actions**
    - Implemented a branch-based CI/CD pipeline using GitHub Actions following industry best practices.
    - The pipeline automates build, test, and deployment processes across multiple environments.
    - Created branches:
        - dev: Installs dependencies, runs Jest tests, builds application
        - stage: Pre-production.
        - prod: Ready to deploy.
    - Created a directory named and .github/workflows and created dev.yml, stage.yml and prod.yml inside the directory in their respective branches.
    - Specified github actions to trigger each branch on commit.
    - Secrets created and used securely with ${{ secrets.* }}.
    - Cached dependencies by specifying them in cache: “npm” in setup node.
    - Integrated email by specifying email_user and email_pass in secrets. Used smtp server to send email when build fails.







- **Git Commands:**
  - **git clone:** to clone a repository to our local machine.

```
kailas@KailasRPillai MINGW64 ~/Downloads
$ git clone https://github.com/Alkaison/Health-Plus.git
Cloning into 'Health-Plus'...
remote: Enumerating objects: 621, done.
remote: Counting objects: 100% (186/186), done.
remote: Compressing objects: 100% (52/52), done.
remote: Total 621 (delta 157), reused 135 (delta 134), pack-reused 435 (from 1)
Receiving objects: 100% (621/621), 4.27 MiB | 1.02 MiB/s, done.
Resolving deltas: 100% (364/364), done.
```

  - **git int:** Creates a new Git repository in the current folder (.git). This converts the current folder into a git repository

```
kailas@KailasRPillai MINGW64 ~/Downloads/Health-Plus (main)
$ git init .
Initialized empty Git repository in C:/Users/kaila/Downloads/Health-Plus/.git/

kailas@KailasRPillai MINGW64 ~/Downloads/Health-Plus (master)
$ git remote add origin https://github.com/KAILAS-R-PILLAI/Health-Plus

kailas@KailasRPillai MINGW64 ~/Downloads/Health-Plus (master)
$ git add .
```

- **git checkout -b branch_name:** to create a switch to a branch.
- **git remote add origin <git_URL>:** Connects the local repo to a remote repo (github) with the url. Origin is the alias given to the git url.
- **git pull origin branch_name:** fetches latest changes from the specific branch.
- **git add filename:** Adds the specific file to git after commit and push. **git add .** will add all files in the repo.
- **git commit -m "commit message":** Done after add and before push to specify what changes have been made as a commit message.
- **git push:** Pushes files into the git repo.

```
kailas@KailasRPillai MINGW64 ~/Downloads/Health-Plus (main)
$ git commit -m "Added CI/CD for react app"
[main e73ce91] Added CI/CD for react app
 1 file changed, 34 insertions(+)
 create mode 100644 .github/workflows/deploy-react.yml

kailas@KailasRPillai MINGW64 ~/Downloads/Health-Plus (main)
$ git push origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 782 bytes | 782.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/KAILAS-R-PILLAI/Health-Plus
   f8ba584..e73ce91  main -> main
```

- **git fetch:** code doesn't change. Updates content in the .git folder. No change to working files. Updates your remote tracking branches.
- **git merge:** Used to combine branches. First switch to a branch and do git merge <branch_name> and it will save changes from that branch to the current branch.
- **git stash:** Temporarily saves changes without commit.

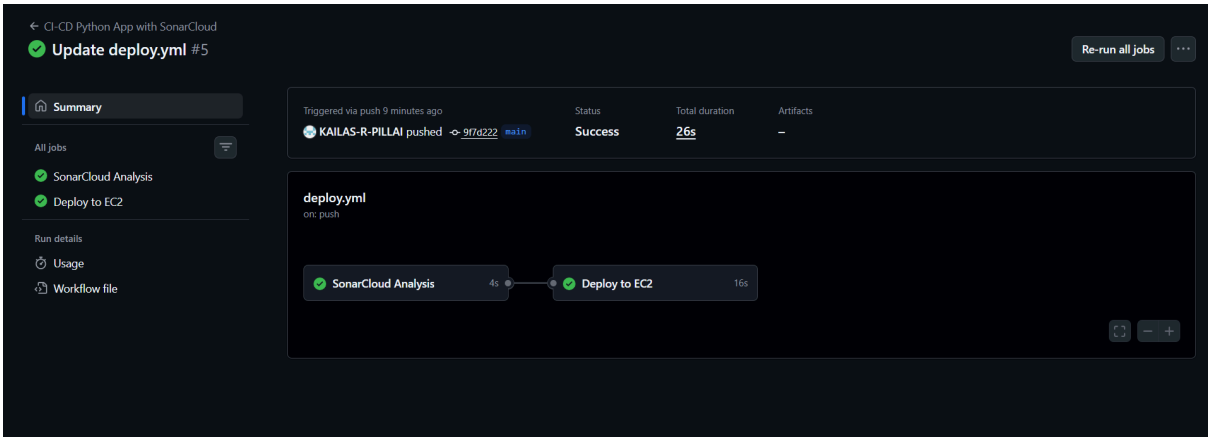
Day 2 - 30/12/2025

Tasks Completed Today:

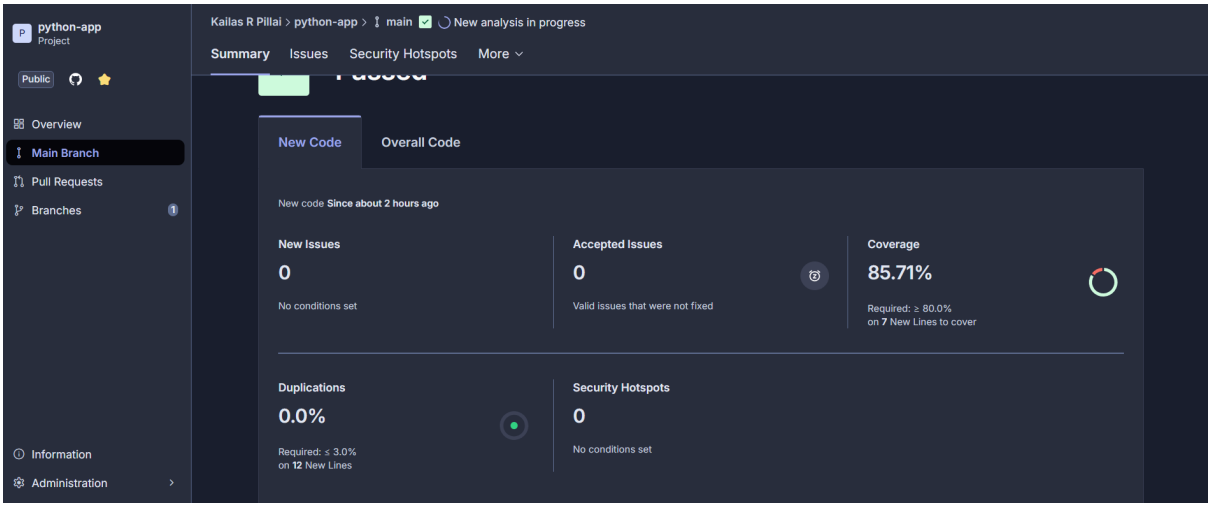
- **Task1:** Learned sonarcloud and integrated with github actions.
- **Task2:** Learned Jenkins, created CI/CD, used caching, used secrets ,sent mail when build fails, created a repo with dev, stage, prod branches, created CI/CD for each branch.

Learnings:

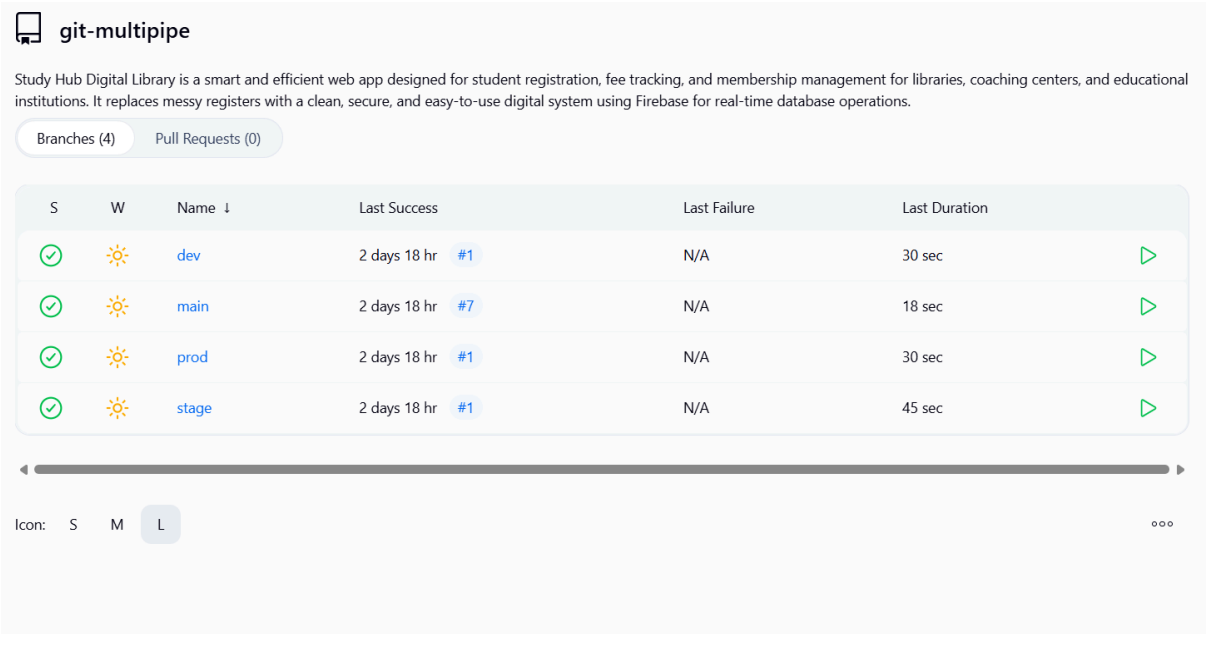
- **Sonarcloud**
  - SonarCloud is a cloud code quality & security checker.
  - It analyzes your code and reports
    - Bugs
    - Code smells (bad practices)
    - Security vulnerabilities
    - Test coverage
    - Duplicated code
  - It runs automatically on every GitHub push / PR.
  - Signed into Sonarcloud via github.
  - Selected the required repo which is a python flask backend application for the new project.
  - Copied the Project key and Organization key and created secret in the secrets and variables tab in the github repository.
  - Defined steps to create virtual environment to install dependency in the deploy.yml.

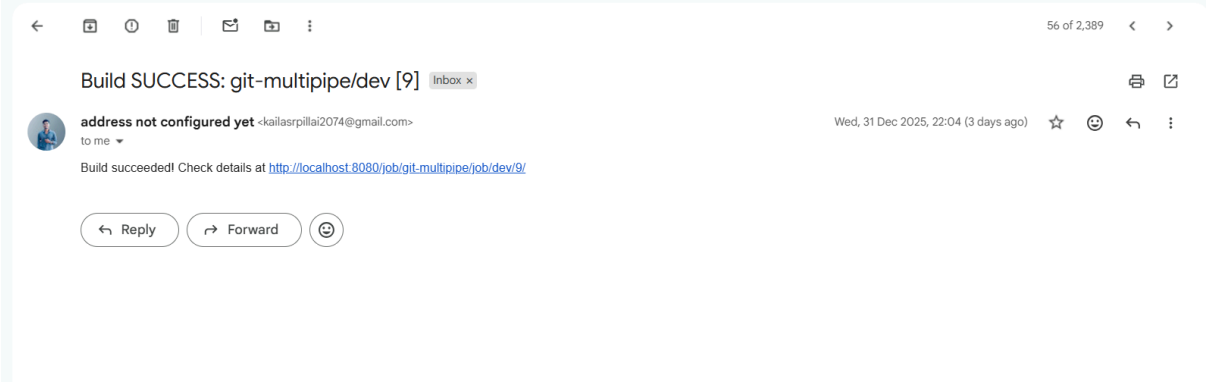


  - Used coverage tools in the github actions and tested the python app.py using unittest.(coverage run -m unittest discover -s tests)
  - Generated coverage.xml to be used by sonarcloud. Defined all these steps in the workflow yaml file.



- **Jenkins**
    - Jenkins is an open-source automation server used to implement CI/CD pipelines by automating build, test, and deployment processes.
    - Written in groovy script.
    - Aim is to create a multi branch pipeline for a github digital library repo.
    - Wrote Jenkinsfile performing the specific repetitive tasks in the dev, stage and prod branches with post actions for success and failure messages to be sent to email.
    - Run Jenkins as a container in docker exposed to 8080.
    - Installed required pipelines for jenkins: Git, NodeJS, Email Extension Plugin
    - Setup email notification credential to be defined in the pipeline.
    - Configured email with the Email extension configuration in jenkins and specified the credential ID for extended email. Set server as smtp.gmail.com and SMTP port to 587.
    - Generated personal access token in github to integrate jenkins and github.
    - Created a multibranch pipeline and specified the git repository URL and applied.
    - Jenkins fetched all the Jenkinsfile in each branch during repository scan.





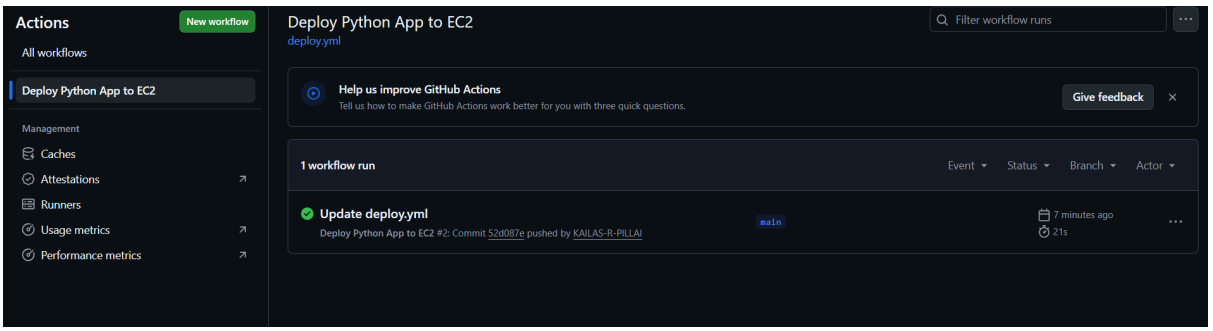
Day 3 - 31/12/2025

Tasks Completed Today:

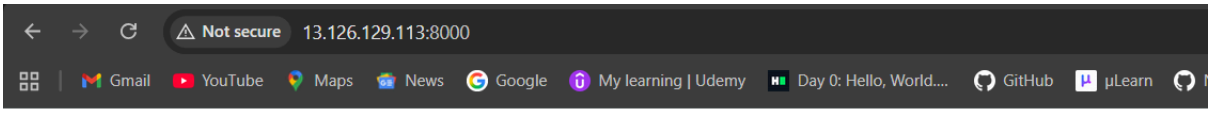
- **Task1:** Created github actions pipeline to deploy nodejs application in bare server.
- **Task2:** Created github actions pipeline to deploy python application in bare server.
- **Task3:** Create github actions pipeline to deploy react application in bare server.
- **Task4:** Integrate sonarcloud with jenkins, learned dependency check and integrate it with jenkins.

Learnings:

- **Github Actions for Application Deployment**
  - Created a python backend flask application and pushed it to github.
  - Wrote deploy.yml for the application inside .github/workflows to deploy the application on EC2 server.
  - Specifies the EC2_USER, EC2_PASSWORD and EC2_SSH_KEY as secrets for logging into the instance.



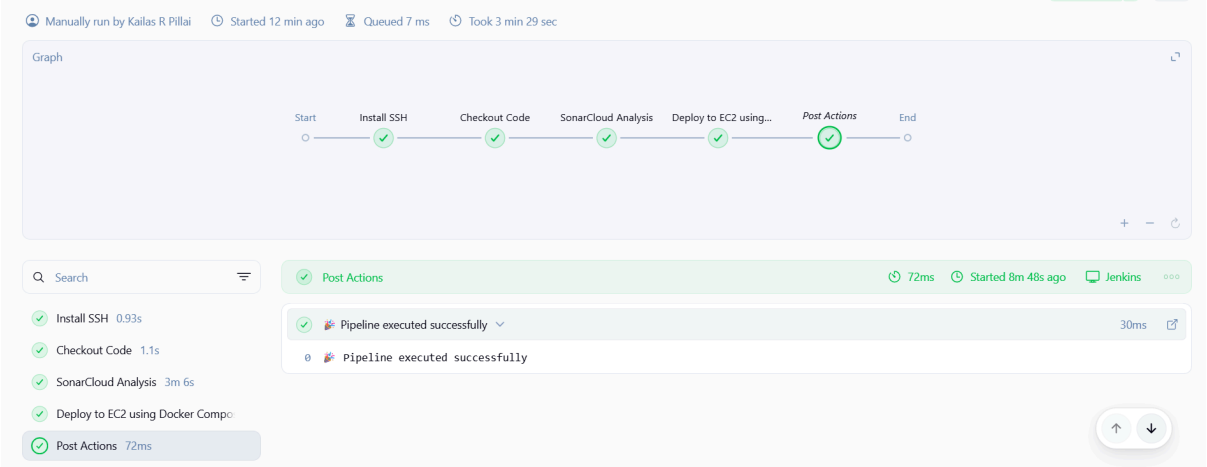
  - Attached elastic IP for static public IP.
  - Set port to 8000.
  - After build success, visited http://ec2_public_ip>:8000 to see the output.



Hello from EC2 🚀

- **Jenkins Sonarcloud Integration**
  - Logged into sonarcloud using github. Selected a Node application as the new project.

- Defined the project key and organization key in the pipeline for the application. Provided the jenkins sonarcloud integration credential.
- Build the pipeline successfully.



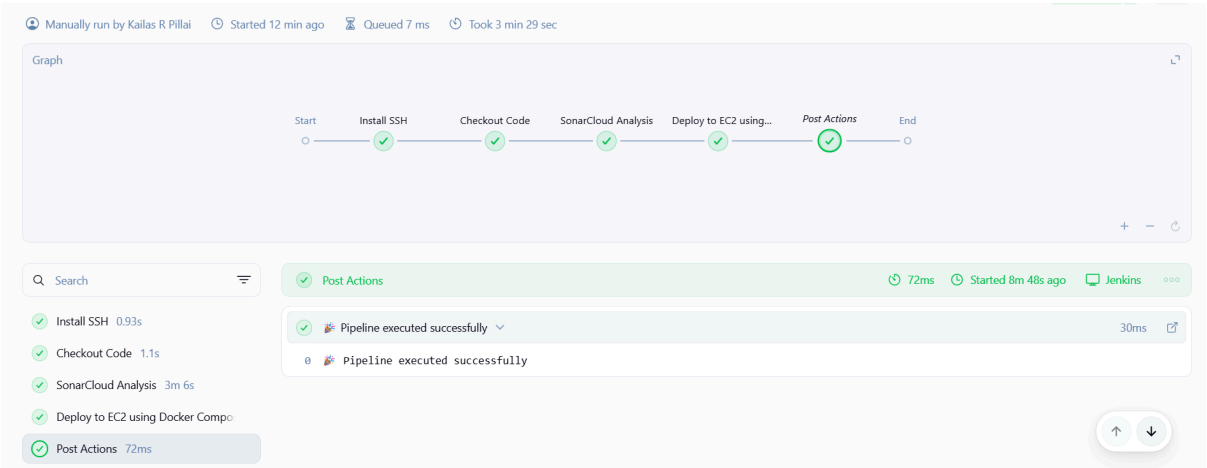
Day 4 - 01/01/2025

Tasks Completed Today:

- **Task1:** Created jenkins pipeline to deploy nodejs application in EC2
- **Task2:** Created jenkins pipeline to deploy python application in EC2
- **Task3:** Created jenkins pipeline to deploy react application in EC2
- **Task4:** Used docker as a agent in jenkins worker nodes

Learnings:

- **Jenkins Pipeline for Application**
  - Created a pipeline that consists of following stages:
    - Install SSH for secure login to EC2
    - Checkout git
    - SonarCloud scan
    - Deploy to EC2 and exposed to port 5000 at ec2 public IP



- Used docker as an agent.
- Jenkins will spin up a temporary Docker container using the node:18-alpine image.
- That container acts as the worker node where all the pipeline steps run.

```
pipeline {
    agent {
        docker {
            image 'node:18-alpine'
            args '-u root'
        }
    }
```

Day 5 - 02/01/2025

Tasks Completed Today:

- **Task1:** Version manager for node application and python
- **Task2:** PM2 process manager

Learnings:

- **Node.js Version Managers**
  - **nvm:** Manage multiple Node.js versions on the same machine.
  - **nvm install 16:** installs latest Node 16.x
  - **nvm use 18:** Shell will run on node 18
  - **node -v:** Shows node version
  - **nvm current:** Shows current active node version
  - **nvm alias default 18:** Sets Node.js version 18 as the default version for all future terminal sessions.

- **Python Version Managers**
  - **pyenv:** Install & switch between Python versions.
  - **pyenv install 3.11.7:** Installs Python version 3.11.7 on the system.
  - **pyenv global 3.11.7:** Sets Python 3.11.7 as the default Python version globally for user.
  - **pyenv local 3.10.12:** Sets Python version for the current project or directory.
  - **python --version:** Checks the active Python version in the terminal session.

- **PM2**
  - PM2 is a production process manager for applications. It helps to run, manage, and monitor applications in the background, making it easier to keep apps alive even after server restarts.

- **PM2 commands**
  - **pm2 start app.js:** Starts the application named app.js
  - **pm2 list:** Lists all running processes
  - **pm2 stop <id>:** Stops the process with the specific ID
  - **pm2 restart <id>:** Restarts the app with the ID
  - **pm2 delete <id>:** Deletes the app with the ID
  - **pm2 logs:** Show logs of all apps
  - **pm2 monit:** Show resource usage
  - **pm2 save:** Save current process list for reboot recovery